CN Set-Wise Lab External Answers

SET-1

a) ifconfig,netstat

Ifconfig :

ifconfig -a

ifconfig enp0s3

ifconfig lo

sudo ifconfig enp0s3 up

sudo ifconfig enp0s3 down

ifconfig -s

Description :

ifconfig – Displays all currently active network interfaces and their configurations.

- **ifconfig -a** – Shows all network interfaces, including inactive ones.
- **ifconfig enp0s3** – Displays detailed configuration of the enp0s3 Ethernet interface.
- **ifconfig lo** – Shows configuration details of the loopback (lo) interface.
- **sudo ifconfig enp0s3 up** – Activates/enables the enp0s3 network interface.
- **sudo ifconfig enp0s3 down** – Deactivates/disables the enp0s3 network interface.
- **ifconfig -s** – Displays a short summary of all interfaces in tabular form.

Netstat :

- **`netstat`** – Displays overall network statistics and all active network connections.
- **`netstat -rn`** – Shows the routing table in numeric format, including destination, gateway, netmask, and interface.
- **`netstat -a`** – Displays all active connections and listening ports.
- **`netstat -t`** – Shows only TCP connections.
- **`netstat -u`** – Shows only UDP connections.
- **`netstat -i`** – Displays statistics of all network interfaces.

b)Write a program to illustrate connection oriented iterative Server

Server Program :

```
#include <stdio.h>
#include <stdlib.h>
 #include <string.h>
 #include <unistd.h>
#include <arpa/inet.h>

int main() {

        int sockfd, newsockfd;
           sockaddr_in serv, cli;
          char buf[50];
          socklen_t clilen;

sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd < 0) { printf("Socket error\n"); exit(1); }

serv.sin_family = AF_INET;
serv.sin_addr.s_addr = htonl(INADDR_ANY);
serv.sin_port = htons(4568);

if (bind(sockfd, (struct sockaddr *)&serv, sizeof(serv)) < 0) {
   printf("Bind error\n"); exit(1);
}

listen(sockfd, 5);
printf("Server waiting… type 'exit' to stop.\n");

clilen = sizeof(cli);

while (1) {
   newsockfd = accept(sockfd, (struct sockaddr *)&cli, &clilen);

   memset(buf, 0, sizeof(buf));
   read(newsockfd, buf, sizeof(buf));
   printf("Received: %s\n", buf);
```

```
    write(newsockfd, buf, sizeof(buf));
    close(newsockfd);

    if (strcmp(buf, "exit") == 0)
        break;
}

printf("Server closed.\n");
close(sockfd);
return 0;


}
```

Client Program :

```
#include <stdio.h>
#include <stdlib.h>
 #include <string.h>
 #include <unistd.h>
 #include <arpa/inet.h>

int main()
{ int sockfd;
 struct sockaddr_in serv;
char msg[50], rcv[50];

sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd < 0) { printf("Socket error\n"); exit(1); }

serv.sin_family = AF_INET;
serv.sin_addr.s_addr = inet_addr("127.0.0.1");
serv.sin_port = htons(4568);

if (connect(sockfd, (struct sockaddr *)&serv, sizeof(serv)) < 0) {
    printf("Connect failed\n"); exit(1);
}

printf("Enter message: ");
```

```
fgets(msg, sizeof(msg), stdin);

write(sockfd, msg, sizeof(msg));
read(sockfd, rcv, sizeof(rcv));

printf("Echo from server: %s\n", rcv);

if (strcmp(rcv, "exit") == 0)
    printf("Closing client\n");

close(sockfd);
return 0;


}
```

SET-2

tcpdump,nslookup

TCPDUMP :

- **`tcpdump --version`** – Checks whether tcpdump is installed on the system.
- **`sudo dnf install -y tcpdump`** – Installs tcpdump using the package manager (Fedora/RHEL).
- **`tcpdump -D`** – Displays all available network interfaces that can be used for packet capture.
- **`sudo tcpdump -i any`** – Captures all packets on all active network interfaces.
- **`sudo tcpdump -i any -c5`** – Captures only 5 packets and then stops automatically.
- **`sudo tcpdump -i any -c5 icmp`** – Captures only ICMP protocol packets.
- **`sudo tcpdump -i any -c5 tcp`** – Captures only TCP protocol packets.

NSLOOKUP :

- **`nslookup domainname`** – Looks up the IP address of a given domain.

- **`nslookup 8.8.8.8`** – Performs a reverse lookup (IP → domain name).
- **`nslookup mvsrec.edu.in`** – Queries DNS to find the IP address for the domain *mvsrec.edu.in*.
- **`nslookup -type=MX domainname`** – Fetches MX (Mail Exchanger) records for a domain.

Write a program to illustrate connection less iterative Server

Server Program :

```
#include <stdlib.h>
 #include <stdio.h>
 #include <sys/socket.h>
 #include <sys/types.h>
 #include <netinet/in.h>
#include <unistd.h>

int main()
{
int sockfd, clilen;
struct sockaddr_in serv_addr, cli_addr; char msg[80];

sockfd = socket(AF_INET, SOCK_DGRAM, 0);
if (sockfd < 0) {
    printf("\nSocket Failed");
    exit(1);
}

serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
serv_addr.sin_port = htons(3456);

if (bind(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) <
0) {
    printf("\nBind Failed");
    exit(1);
}
```

```
clilen = sizeof(cli_addr);
printf("Server waiting for message...\n");

recvfrom(sockfd, msg, sizeof(msg), 0, (struct sockaddr *)&cli_addr,
&clilen);
printf("\nServer Received: %s\n", msg);

sendto(sockfd, msg, sizeof(msg), 0, (struct sockaddr *)&cli_addr,
clilen);

close(sockfd);
return 0;}
```

Client Program :

```
#include <stdlib.h>
 #include <stdio.h>
#include <sys/types.h>
#include <netinet/in.h>
 #include <string.h>
#include <sys/socket.h>
#include <unistd.h>

int main()
{ int sockfd, n, servlen;
struct sockaddr_in cli_addr, serv_addr ;
char msg[50], msg1[50] ;

sockfd = socket(AF_INET, SOCK_DGRAM, 0);
if (sockfd < 0) {
    printf("\nSocket Failed");
    exit(1);
}

serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = inet_addr("192.168.2.58");   // Server IP
serv_addr.sin_port = htons(3456);

cli_addr.sin_family = AF_INET;
```

```
cli_addr.sin_addr.s_addr = htonl(INADDR_ANY);
cli_addr.sin_port = htons(0);

if (bind(sockfd, (struct sockaddr *)&cli_addr, sizeof(cli_addr)) < 0)
{
    printf("\nClient can't bind");
    exit(1);
}

printf("Enter String: ");
fgets(msg, 50, stdin);

if (sendto(sockfd, msg, 50, 0, (struct sockaddr *)&serv_addr,
sizeof(serv_addr)) < 0) {
    printf("\nClient sendto error");
    exit(1);
}

servlen = sizeof(serv_addr);
n = recvfrom(sockfd, msg1, 50, 0, (struct sockaddr *)&serv_addr,
&servlen);

if (n < 0) {
    printf("\nReceive error");
    exit(1);
} else {
    printf("\nClient received msg: %s\n", msg1);
}

close(sockfd);
return 0;


}
```

SET-3

traceroute,FTP

Traceroute :

sudo apt install traceroute – Installs the traceroute tool.

- **traceroute <hostname/IP>** – Traces the path packets take to the destination.
- **traceroute google.com** – Shows all hops between your system and google.com.
- **traceroute -n <IP>** – Shows only numeric IPs (skips DNS lookup).
- **traceroute -m 20 <IP>** – Sets maximum hop count to 20.
- **traceroute -w 2 <IP>** – Sets a 2-second timeout for each hop.

FTP :

ftp [IP] – Connect to a remote FTP server using its IP address.

- **ftp 192.168.100.9** – Example command to connect to the FTP server at 192.168.100.9.
- **(Enter Username)** – Provide the FTP username when prompted.
- **(Enter Password)** – Provide the FTP password to log in.

Write a program to illustrate connection oriented concurrent Server

Server Program :

```
#include <stdio.h>
#include <stdlib.h>
 #include <string.h>
 #include <unistd.h>
#include <arpa/inet.h>

int main() {

        int sockfd, newsockfd;
           sockaddr_in serv, cli;
          char buf[50];
          socklen_t clilen;

sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd < 0) { printf("Socket error\n"); exit(1); }
```

```
serv.sin_family = AF_INET;
serv.sin_addr.s_addr = htonl(INADDR_ANY);
serv.sin_port = htons(4568);

if (bind(sockfd, (struct sockaddr *)&serv, sizeof(serv)) < 0) {
   printf("Bind error\n"); exit(1);
}

listen(sockfd, 5);
printf("Server waiting… type 'exit' to stop.\n");

clilen = sizeof(cli);

while (1) {
   newsockfd = accept(sockfd, (struct sockaddr *)&cli, &clilen);

   memset(buf, 0, sizeof(buf));
   read(newsockfd, buf, sizeof(buf));
   printf("Received: %s\n", buf);

   write(newsockfd, buf, sizeof(buf));
   close(newsockfd);

   if (strcmp(buf, "exit") == 0)
      break;
}

printf("Server closed.\n");
close(sockfd);
return 0;


}
```

Client Program :

```
#include <stdio.h>
#include <stdlib.h>
 #include <string.h>
```

```
 #include <unistd.h>
 #include <arpa/inet.h>

int main()
{ int sockfd;
 struct sockaddr_in serv;
char msg[50], rcv[50];

sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd < 0) { printf("Socket error\n"); exit(1); }

serv.sin_family = AF_INET;
serv.sin_addr.s_addr = inet_addr("127.0.0.1");
serv.sin_port = htons(4568);

if (connect(sockfd, (struct sockaddr *)&serv, sizeof(serv)) < 0) {
    printf("Connect failed\n"); exit(1);
}

printf("Enter message: ");
fgets(msg, sizeof(msg), stdin);

write(sockfd, msg, sizeof(msg));
read(sockfd, rcv, sizeof(rcv));

printf("Echo from server: %s\n", rcv);

if (strcmp(rcv, "exit") == 0)
    printf("Closing client\n");

close(sockfd);
return 0;


}
```

SET-4

telnet,ping

Telnet Commands :

- **telnet <hostname>** – Connects to a remote host using the default port 23.
- **telnet google.com 80** – Connects to google.com on port 80 (HTTP) to check if the port is open.
- **telnet smtp.gmail.com 25** – Connects to Gmail's SMTP server on port 25 for testing mail server connectivity.

Ping Commands :

- **ping <domain_or_IP>** – Tests the reachability of a host and measures round-trip time.
- **ping -c 5 <domain_or_IP>** – Sends 5 ping packets to the host and then stops.

Write a program to illustrate connection less concurrent Server

Server Program :

```
#include <stdlib.h>
 #include <stdio.h>
 #include <sys/socket.h>
 #include <sys/types.h>
 #include <netinet/in.h>
#include <unistd.h>

int main()
 {
int sockfd, clilen;
struct sockaddr_in serv_addr, cli_addr; char msg[80];

sockfd = socket(AF_INET, SOCK_DGRAM, 0);
if (sockfd < 0) {
   printf("\nSocket Failed");
   exit(1);
}

serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
serv_addr.sin_port = htons(3456);

if (bind(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
   printf("\nBind Failed");
```

```
    exit(1);
}

clilen = sizeof(cli_addr);
printf("Server waiting for message...\n");

recvfrom(sockfd, msg, sizeof(msg), 0, (struct sockaddr *)&cli_addr, &clilen);
printf("\nServer Received: %s\n", msg);

sendto(sockfd, msg, sizeof(msg), 0, (struct sockaddr *)&cli_addr, clilen);

close(sockfd);
return 0; }
```

Client Program :

```
#include <stdlib.h>
 #include <stdio.h>
#include <sys/types.h>
#include <netinet/in.h>
 #include <string.h>
#include <sys/socket.h>
#include <unistd.h>

int main()
 { int sockfd, n, servlen;
struct sockaddr_in cli_addr, serv_addr ;
char msg[50], msg1[50] ;

sockfd = socket(AF_INET, SOCK_DGRAM, 0);
if (sockfd < 0) {
    printf("\nSocket Failed");
    exit(1);
}

serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = inet_addr("192.168.2.58");   // Server IP
serv_addr.sin_port = htons(3456);

cli_addr.sin_family = AF_INET;
```

```
cli_addr.sin_addr.s_addr = htonl(INADDR_ANY);
cli_addr.sin_port = htons(0);

if (bind(sockfd, (struct sockaddr *)&cli_addr, sizeof(cli_addr)) < 0) {
    printf("\nClient can't bind");
    exit(1);
}

printf("Enter String: ");
fgets(msg, 50, stdin);

if (sendto(sockfd, msg, 50, 0, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
    printf("\nClient sendto error");
    exit(1);
}

servlen = sizeof(serv_addr);
n = recvfrom(sockfd, msg1, 50, 0, (struct sockaddr *)&serv_addr, &servlen);

if (n < 0) {
    printf("\nReceive error");
    exit(1);
} else {
    printf("\nClient received msg: %s\n", msg1);
}

close(sockfd);
return 0;
}
```

SET—5

ifconfig,netstat.

Ifconfig :

ifconfig -a

ifconfig enp0s3

ifconfig lo

sudo ifconfig enp0s3 up

sudo ifconfig enp0s3 down

ifconfig -s

Description :

ifconfig – Displays all currently active network interfaces and their configurations.

- **ifconfig -a** – Shows all network interfaces, including inactive ones.
- **ifconfig enp0s3** – Displays detailed configuration of the enp0s3 Ethernet interface.
- **ifconfig lo** – Shows configuration details of the loopback (lo) interface.
- **sudo ifconfig enp0s3 up** – Activates/enables the enp0s3 network interface.
- **sudo ifconfig enp0s3 down** – Deactivates/disables the enp0s3 network interface.
- **ifconfig -s** – Displays a short summary of all interfaces in tabular form.

Netstat :

- **`netstat`** – Displays overall network statistics and all active network connections.
- **`netstat -rn`** – Shows the routing table in numeric format, including destination, gateway, netmask, and interface.
- **`netstat -a`** – Displays all active connections and listening ports.
- **`netstat -t`** – Shows only TCP connections.
- **`netstat -u`** – Shows only UDP connections.

`netstat -i` – Displays statistics of all network interfaces.

Write a program to illustrate date time client/Server.

Date Time Server :

```
#include <stdio.h>
 #include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
 #include <time.h>
```

```
int main()
 { int sockfd, newsockfd;
 struct sockaddr_in serv_addr, cli_addr;
socklen_t clilen;
char buf[50];
time_t now; struct tm present;

sockfd = socket(AF_INET, SOCK_STREAM, 0);
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
serv_addr.sin_port = htons(3100);

bind(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
listen(sockfd, 5);
clilen = sizeof(cli_addr);

while (1) {
   newsockfd = accept(sockfd, (struct sockaddr *)&cli_addr, &clilen);
   memset(buf, 0, sizeof(buf));
   read(newsockfd, buf, sizeof(buf));

   time(&now);
   present = *localtime(&now);
   sprintf(buf, "Time: %d-%d-%d %d:%d:%d\n",
         present.tm_year + 1900, present.tm_mon + 1, present.tm_mday,
         present.tm_hour, present.tm_min, present.tm_sec);

   write(newsockfd, buf, sizeof(buf));
   close(newsockfd);
}
return 0;


}
```

Date Time Client :

```
#include <stdio.h>
 #include <stdlib.h>
 #include <string.h>
```

```
 #include <unistd.h>
 #include <arpa/inet.h>
#include <sys/socket.h>

int main()
 { int sockfd;
 struct sockaddr_in serv_addr;
char buf[50];

sockfd = socket(AF_INET, SOCK_STREAM, 0);
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
serv_addr.sin_port = htons(3100);

connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
write(sockfd, buf, sizeof(buf));
read(sockfd, buf, sizeof(buf));

printf("Client Received: %s\n", buf);
close(sockfd);
return 0;

}
```

SET—6

tcpdump,nslookup

TCPDUMP :

- **`tcpdump --version`** – Checks whether tcpdump is installed on the system.
- **`sudo dnf install -y tcpdump`** – Installs tcpdump using the package manager (Fedora/RHEL).
- **`tcpdump -D`** – Displays all available network interfaces that can be used for packet capture.
- **`sudo tcpdump -i any`** – Captures all packets on all active network interfaces.
- **`sudo tcpdump -i any -c5`** – Captures only 5 packets and then stops automatically.
- **`sudo tcpdump -i any -c5 icmp`** – Captures only ICMP protocol packets.
- **`sudo tcpdump -i any -c5 tcp`** – Captures only TCP protocol packets.

NSLOOKUP :

- **`nslookup domainname`** – Looks up the IP address of a given domain.
- **`nslookup 8.8.8.8`** – Performs a reverse lookup (IP → domain name).
- **`nslookup mvsrec.edu.in`** – Queries DNS to find the IP address for the domain *mvsrec.edu.in*.
- **`nslookup -type=MX domainname`** – Fetches MX (Mail Exchanger) records for a domain.

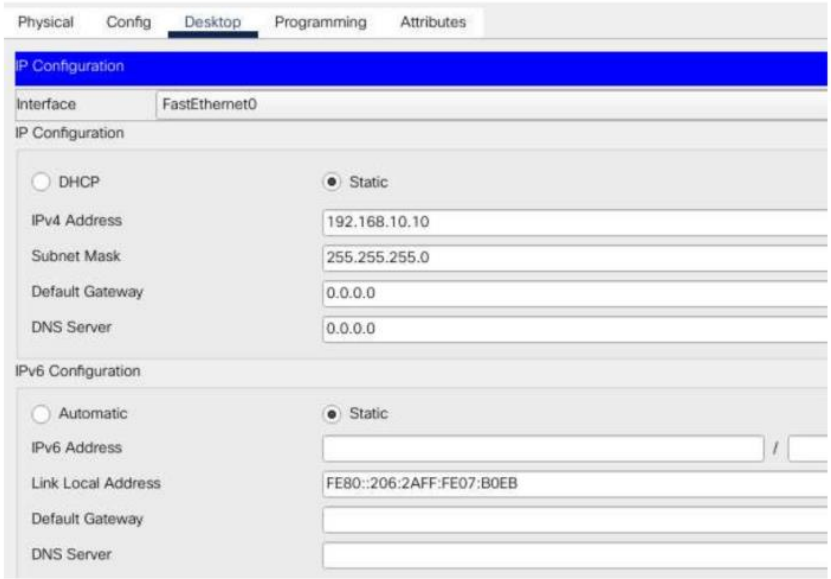
Using CPT to connect to end systems and establish communication between them.

Example 1: Using CPT to connect to end systems and establish communication between them.

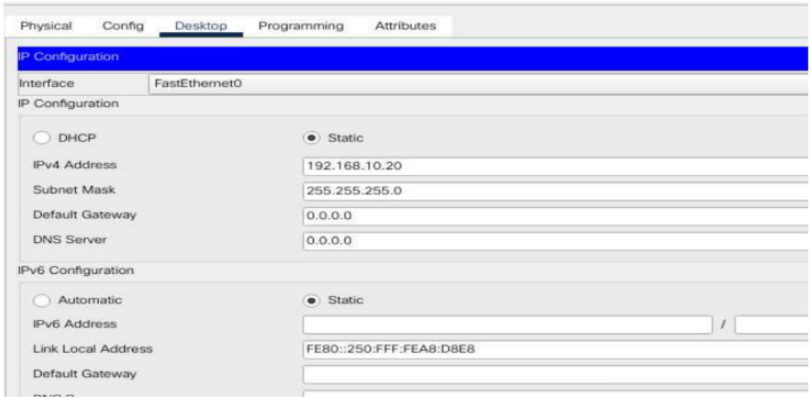
Step 1: Add two end devices on to the work space and connect them using automatic cable

Step 2: Click on computer so that u get:

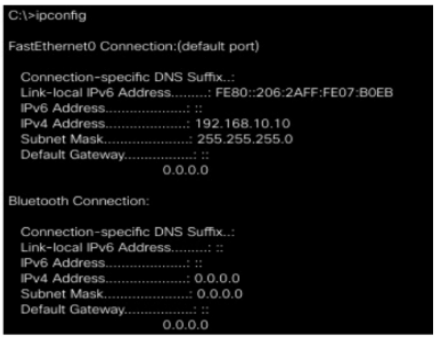
Step 3: In Desktop tab, select IP Configuration and give 192.168.10.10 in IPv4 address text box and press tab



Do the same for the second pc, with IP: 192.168.10.20 and press tab.



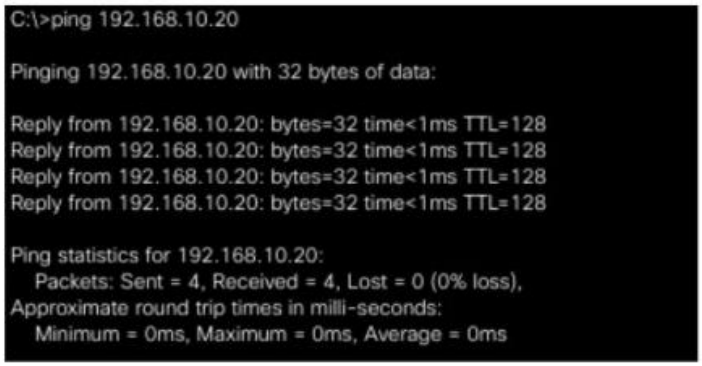
Step 4: To see the ip address, type **ipconfig** in the command prompt of the host:



To see MAC/PHYSICAL ADDRESS: TYPE: ipconfig /all

Step 5: to check for the communication established or not type

Ping 192.168.10.20 (you should type this command in host 1: 192.168.10.10)



SET—7

Traceroute,FTP

Traceroute :

sudo apt install traceroute – Installs the traceroute tool.

- **traceroute <hostname/IP>** – Traces the path packets take to the destination.
- **traceroute google.com** – Shows all hops between your system and google.com.
- **traceroute -n <IP>** – Shows only numeric IPs (skips DNS lookup).
- **traceroute -m 20 <IP>** – Sets maximum hop count to 20.
- **traceroute -w 2 <IP>** – Sets a 2-second timeout for each hop.

FTP :

ftp [IP] – Connect to a remote FTP server using its IP address.

- **ftp 192.168.100.9** – Example command to connect to the FTP server at 192.168.100.9.
- **(Enter Username)** – Provide the FTP username when prompted.
- **(Enter Password)** – Provide the FTP password to log in.

Using CPT to configure router to establish connection between two different networks.

- **Step 1:** Add devices
  - Add PCs, switches (2960), and a router (1941).
  - Connect them to form two networks.
- **Step 2:** Assign IP addresses to end devices
  - **Network 1:** PC0 → 192.168.10.10, PC1 → 192.168.10.20
  - **Network 2:** PC2 → 192.168.20.10, PC3 → 192.168.20.20
- **Step 3:** Open Router CLI
  - Click on the router and go to the CLI tab.
- **Step 4:** Skip initial configuration dialog
  - Type n when prompted and press Enter.
  - You will be in **User EXEC Mode** (> prompt).
- **Step 5:** Enter Privileged EXEC Mode
  - Type `enable`.
  - Now in **Privileged Mode** (# prompt).
- **Step 6:** View current configuration
  - Type `show running-config`.
  - Displays the current router settings.
- **Step 7:** Enter Global Configuration Mode
  - Type `configure terminal`.

    - Now in **Global Configuration Mode**.
- **Step 8:** Enter interface configuration mode
    - Type `interface gigabitEthernet 0/0`.
- **Step 9:** Assign IP address to interface
    - Type `ip address 192.168.10.100 255.255.255.0`.
- **Step 10:** Enable the interface
    - Type `no shutdown`.
    - Interface turns from administratively down (red) to up (green).
- **Step 11:** Exit interface configuration mode
    - Type `exit` to return to Privileged EXEC Mode.
- **Step 12:** Repeat for the second interface
    - Type `interface gigabitEthernet 0/1`.
    - Assign IP address and `no shutdown`.

SET— 8

telnet,ping

Telnet Commands :

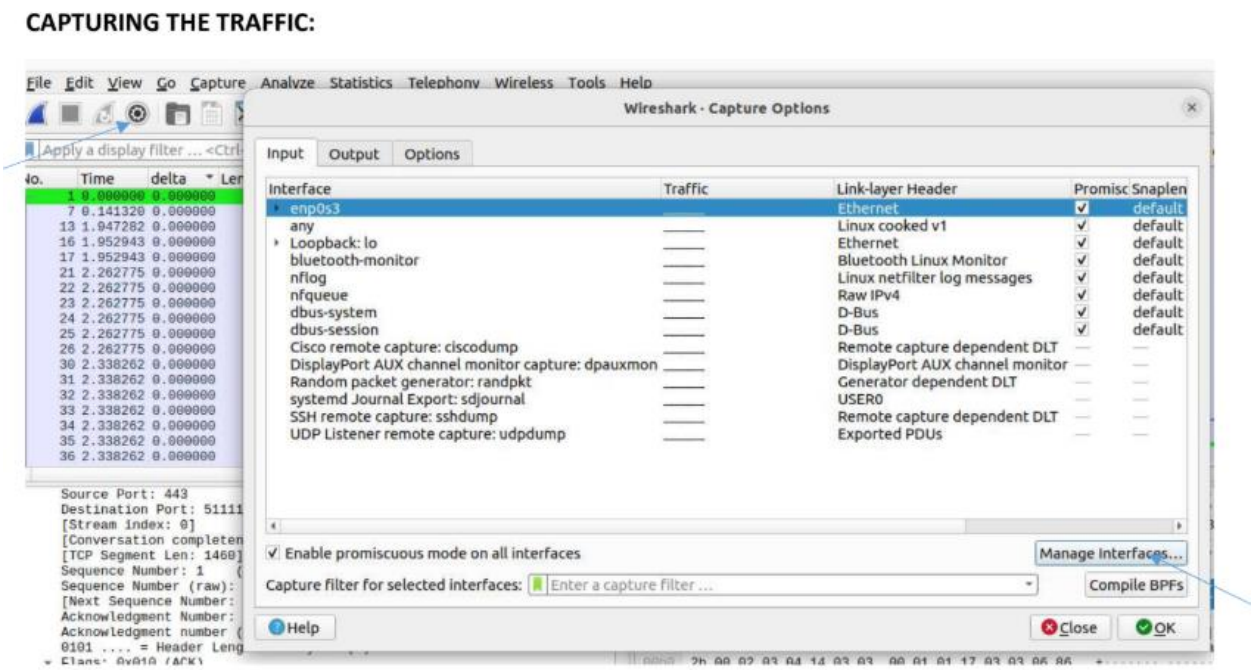
- **telnet <hostname>** – Connects to a remote host using the default port 23.
- **telnet google.com 80** – Connects to google.com on port 80 (HTTP) to check if the port is open.
- **telnet smtp.gmail.com 25** – Connects to Gmail's SMTP server on port 25 for testing mail server connectivity.

Ping Commands :

- **ping <domain_or_IP>** – Tests the reachability of a host and measures round-trip time.

ping -c 5 <domain_or_IP> – Sends 5 ping packets to the host and then stops

CAPTURING THE TRAFFIC using wireshark



SET— 9

Write a program to illustrate date time client/Server.

Date Time Server :

```
#include <stdio.h>
 #include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
 #include <time.h>

int main()
 { int sockfd, newsockfd;
 struct sockaddr_in serv_addr, cli_addr;
socklen_t clilen;
char buf[50];
time_t now; struct tm present;
```

```
sockfd = socket(AF_INET, SOCK_STREAM, 0);
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
serv_addr.sin_port = htons(3100);

bind(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
listen(sockfd, 5);
clilen = sizeof(cli_addr);

while (1) {
   newsockfd = accept(sockfd, (struct sockaddr *)&cli_addr, &clilen);
   memset(buf, 0, sizeof(buf));
   read(newsockfd, buf, sizeof(buf));

   time(&now);
   present = *localtime(&now);
   sprintf(buf, "Time: %d-%d-%d %d:%d:%d\n",
         present.tm_year + 1900, present.tm_mon + 1, present.tm_mday,
         present.tm_hour, present.tm_min, present.tm_sec);

   write(newsockfd, buf, sizeof(buf));
   close(newsockfd);
}
return 0;


}
```

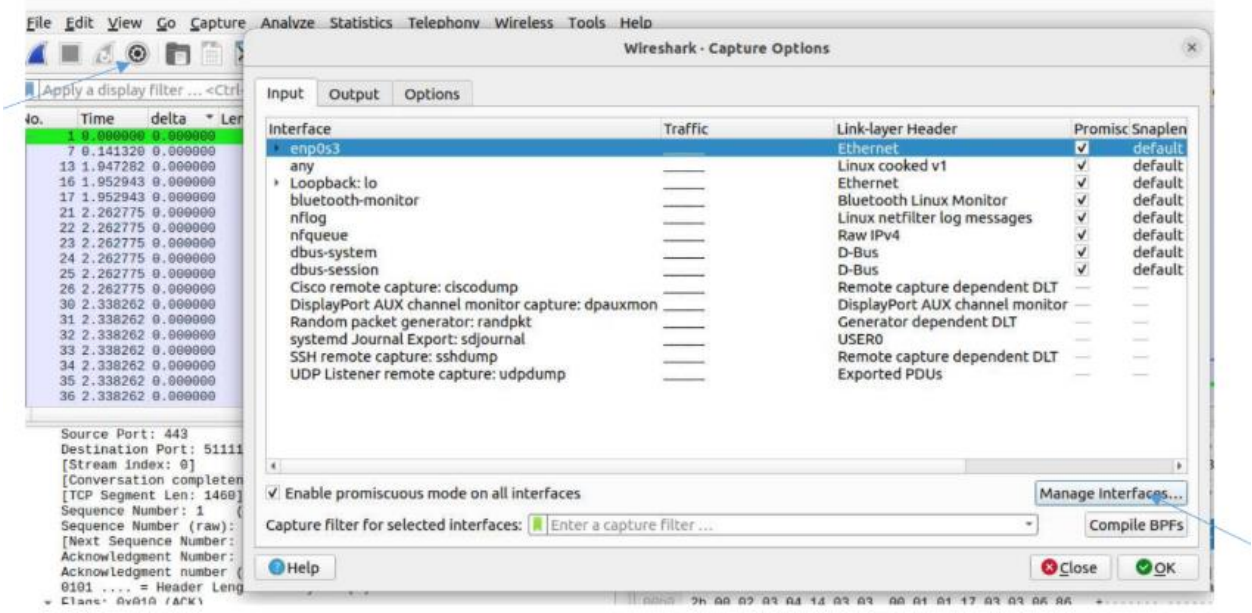
Date Time Client :

```
#include <stdio.h>
 #include <stdlib.h>
 #include <string.h>
 #include <unistd.h>
 #include <arpa/inet.h>
#include <sys/socket.h>

int main()
 { int sockfd;
```

```
 struct sockaddr_in serv_addr;
char buf[50];

sockfd = socket(AF_INET, SOCK_STREAM, 0);
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
serv_addr.sin_port = htons(3100);

connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
write(sockfd, buf, sizeof(buf));
read(sockfd, buf, sizeof(buf));

printf("Client Received: %s\n", buf);
close(sockfd);
return 0;


}
```

CAPTURING THE TRAFFIC using wireshark

CAPTURING THE TRAFFIC:



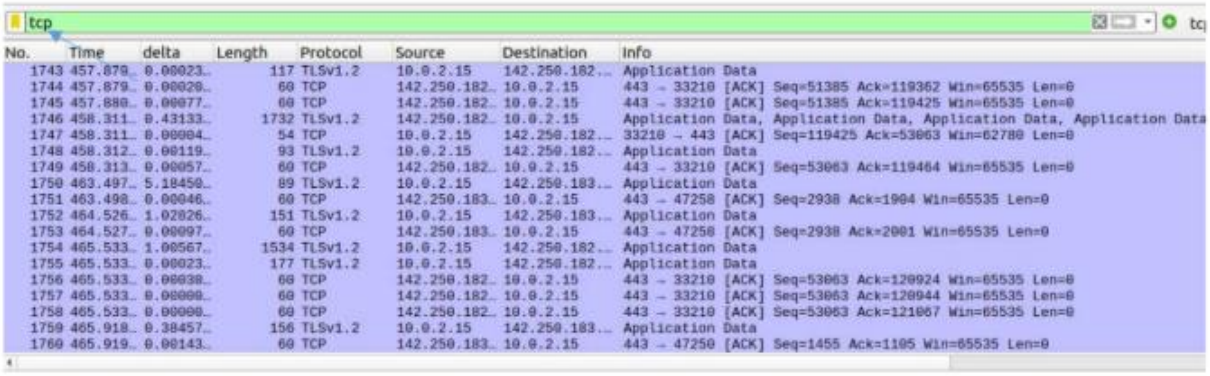
SET—10

Program to implement DNS

```
#include<stdio.h>
 #include<netdb.h>
 #include<arpa/inet.h>
#include<netinet/in.h>
 #include<stdlib.h>

int main(int argc, char** argv)
 { struct hostent *host;
struct in_addr h_addr;

if (argc != 2)
{
   fprintf(stderr, "USAGE: nslookup <hostname>\n");
   exit(1);
}

if ((host = gethostbyname(argv[1])) == NULL)
{
   fprintf(stderr, "(mini)nslookup failed on %s\n", argv[1]);
   exit(1);
}

h_addr.s_addr = *((unsigned long*)host->h_addr_list[0]);
printf("\nIP ADDRESS = %s\n", inet_ntoa(h_addr));
printf("HOST NAME = %s\n", host->h_name);
printf("ADDRESS LENGTH = %d\n", host->h_length);
printf("ADDRESS TYPE = %d\n", host->h_addrtype);

return 0;
 }
```

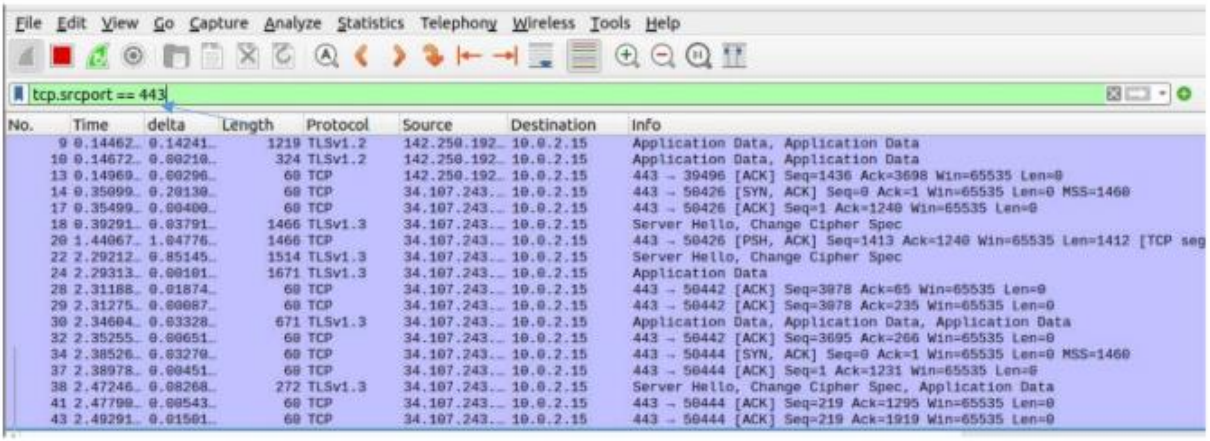
FILTERING TRAFFIC using wireshark.

FILTERING TRAFFIC:



tcp

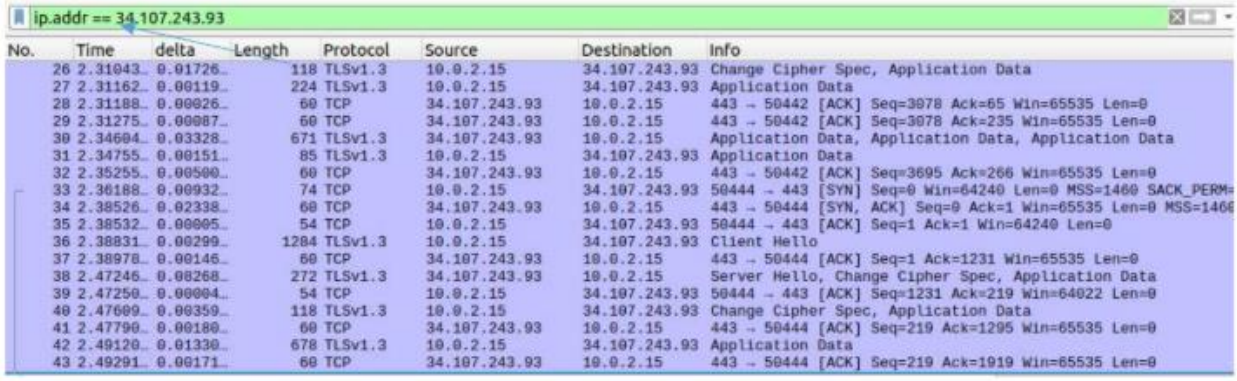
No.	Time	delta	Length	Protocol	Source	Destination	Info
1743	457.879…	0.00023…	117	TLSv1.2	10.0.2.15	142.250.182…	Application Data
1744	457.879…	0.00020…	60	TCP	142.250.182…	10.0.2.15	443 → 33210 [ACK] Seq=51385 Ack=119362 Win=65535 Len=0
1745	457.880…	0.00077…	60	TCP	142.250.182…	10.0.2.15	443 → 33210 [ACK] Seq=51385 Ack=119425 Win=65535 Len=0
1746	458.311…	0.43133…	1732	TLSv1.2	142.250.182…	10.0.2.15	Application Data, Application Data, Application Data, Application Data
1747	458.311…	0.00004…	54	TCP	10.0.2.15	142.250.182…	33210 → 443 [ACK] Seq=119425 Ack=53063 Win=62780 Len=0
1748	458.312…	0.00119…	93	TLSv1.2	10.0.2.15	142.250.182…	Application Data
1749	458.313…	0.00057…	60	TCP	142.250.182…	10.0.2.15	443 → 33210 [ACK] Seq=53063 Ack=119464 Win=65535 Len=0
1750	463.497…	5.18450…	89	TLSv1.2	10.0.2.15	142.250.183…	Application Data
1751	463.498…	0.00046…	60	TCP	142.250.183…	10.0.2.15	443 → 47258 [ACK] Seq=2938 Ack=1904 Win=65535 Len=0
1752	464.526…	1.02826…	151	TLSv1.2	10.0.2.15	142.250.183…	Application Data
1753	464.527…	0.00097…	60	TCP	142.250.183…	10.0.2.15	443 → 47258 [ACK] Seq=2938 Ack=2001 Win=65535 Len=0
1754	465.533…	1.00567…	1534	TLSv1.2	10.0.2.15	142.250.182…	Application Data
1755	465.533…	0.00023…	177	TLSv1.2	10.0.2.15	142.250.182…	Application Data
1756	465.533…	0.00038…	60	TCP	142.250.182…	10.0.2.15	443 → 33210 [ACK] Seq=53063 Ack=120924 Win=65535 Len=0
1757	465.533…	0.00000…	60	TCP	142.250.182…	10.0.2.15	443 → 33210 [ACK] Seq=53063 Ack=120944 Win=65535 Len=0
1758	465.533…	0.00000…	60	TCP	142.250.182…	10.0.2.15	443 → 33210 [ACK] Seq=53063 Ack=121067 Win=65535 Len=0
1759	465.918…	0.38457…	156	TLSv1.2	10.0.2.15	142.250.183…	Application Data
1760	465.919…	0.00143…	60	TCP	142.250.183…	10.0.2.15	443 → 47258 [ACK] Seq=1455 Ack=1105 Win=65535 Len=0



File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

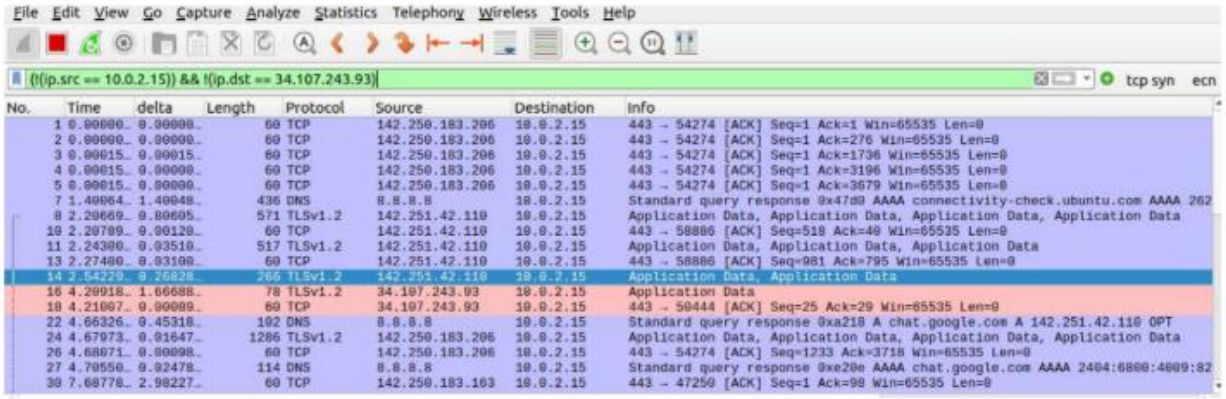
tcp.srcport == 443

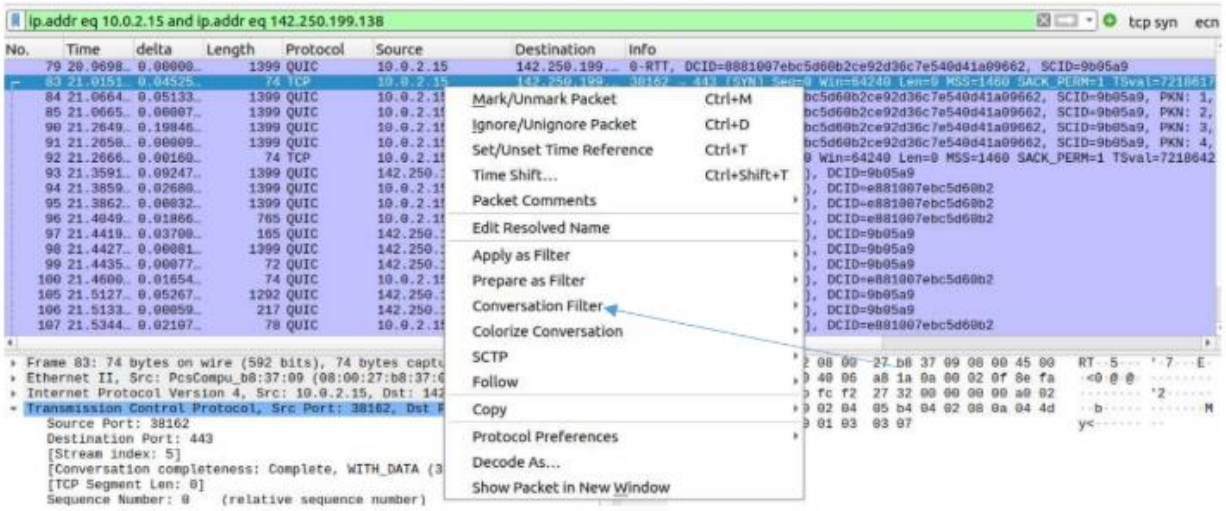
No.	Time	delta	Length	Protocol	Source	Destination	Info
9	0.14462…	0.14241…	1219	TLSv1.2	142.250.192…	10.0.2.15	Application Data, Application Data
10	0.14672…	0.00210…	324	TLSv1.2	142.250.192…	10.0.2.15	Application Data, Application Data
13	0.14969…	0.00296…	60	TCP	142.250.192…	10.0.2.15	443 → 39496 [ACK] Seq=1436 Ack=3698 Win=65535 Len=0
14	0.35099…	0.20130…	60	TCP	34.107.243…	10.0.2.15	443 → 50426 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
17	0.35499…	0.00400…	60	TCP	34.107.243…	10.0.2.15	443 → 50426 [ACK] Seq=1 Ack=1240 Win=65535 Len=0
18	0.39291…	0.03791…	1466	TLSv1.3	34.107.243…	10.0.2.15	Server Hello, Change Cipher Spec
20	1.44067…	1.04776…	1466	TCP	34.107.243…	10.0.2.15	443 → 50426 [PSH, ACK] Seq=1413 Ack=1240 Win=65535 Len=1412 [TCP seg
22	2.29212…	0.85145…	1514	TLSv1.3	34.107.243…	10.0.2.15	Server Hello, Change Cipher Spec
24	2.29313…	0.00101…	1671	TLSv1.3	34.107.243…	10.0.2.15	Application Data
28	2.31188…	0.01874…	60	TCP	34.107.243…	10.0.2.15	443 → 50442 [ACK] Seq=3078 Ack=65 Win=65535 Len=0
29	2.31275…	0.00087…	60	TCP	34.107.243…	10.0.2.15	443 → 50442 [ACK] Seq=3078 Ack=235 Win=65535 Len=0
30	2.34604…	0.03328…	671	TLSv1.3	34.107.243…	10.0.2.15	Application Data, Application Data, Application Data
32	2.35255…	0.00651…	60	TCP	34.107.243…	10.0.2.15	443 → 50442 [ACK] Seq=3695 Ack=266 Win=65535 Len=0
34	2.38526…	0.03270…	60	TCP	34.107.243…	10.0.2.15	443 → 50444 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
37	2.38978…	0.00451…	60	TCP	34.107.243…	10.0.2.15	443 → 50444 [ACK] Seq=1 Ack=1231 Win=65535 Len=0
38	2.47246…	0.08268…	272	TLSv1.3	34.107.243…	10.0.2.15	Server Hello, Change Cipher Spec, Application Data
41	2.47790…	0.00543…	60	TCP	34.107.243…	10.0.2.15	443 → 50444 [ACK] Seq=219 Ack=1295 Win=65535 Len=0
43	2.49291…	0.01501…	60	TCP	34.107.243…	10.0.2.15	443 → 50444 [ACK] Seq=219 Ack=1919 Win=65535 Len=0

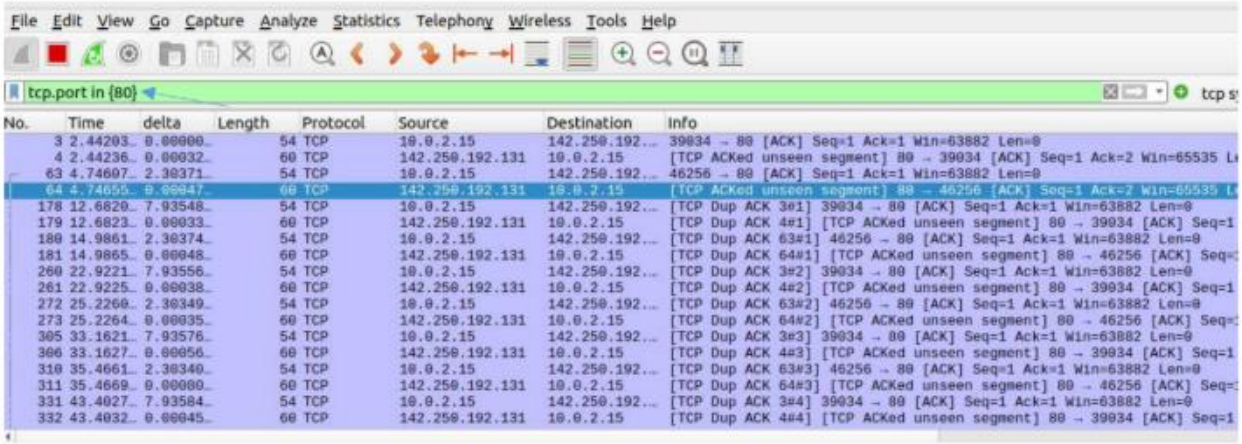


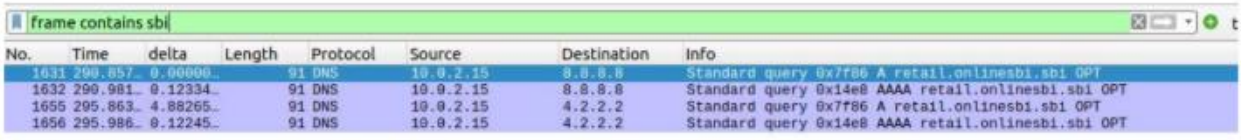
ip.addr == 34.107.243.93

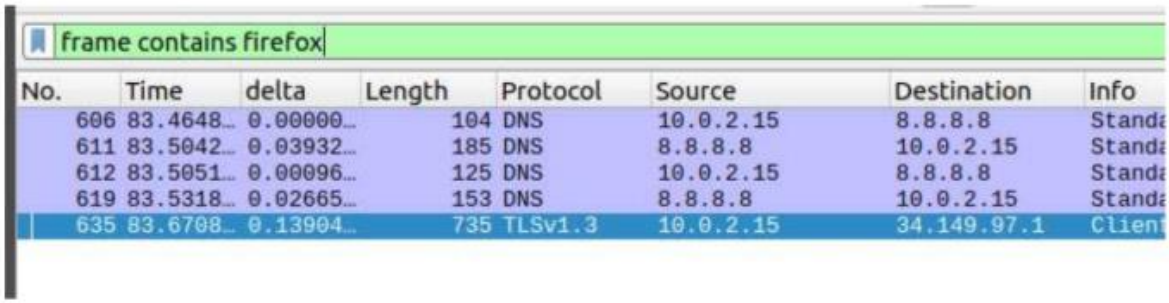
No.	Time	delta	Length	Protocol	Source	Destination	Info
26	2.31043…	0.01726…	118	TLSv1.3	10.0.2.15	34.107.243.93	Change Cipher Spec, Application Data
27	2.31162…	0.00119…	224	TLSv1.3	10.0.2.15	34.107.243.93	Application Data
28	2.31188…	0.00026…	60	TCP	34.107.243.93	10.0.2.15	443 → 50442 [ACK] Seq=3078 Ack=65 Win=65535 Len=0
29	2.31275…	0.00087…	60	TCP	34.107.243.93	10.0.2.15	443 → 50442 [ACK] Seq=3078 Ack=235 Win=65535 Len=0
30	2.34604…	0.03328…	671	TLSv1.3	34.107.243.93	10.0.2.15	Application Data, Application Data, Application Data
31	2.34755…	0.00151…	85	TLSv1.3	10.0.2.15	34.107.243.93	Application Data
32	2.35255…	0.00500…	60	TCP	34.107.243.93	10.0.2.15	443 → 50442 [ACK] Seq=3695 Ack=266 Win=65535 Len=0
33	2.36188…	0.00932…	74	TCP	10.0.2.15	34.107.243.93	50444 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=
34	2.38526…	0.02338…	60	TCP	34.107.243.93	10.0.2.15	443 → 50444 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=146
35	2.38532…	0.00005…	54	TCP	10.0.2.15	34.107.243.93	50444 → 443 [ACK] Seq=1 Ack=1 Win=64240 Len=0
36	2.38831…	0.00299…	1284	TLSv1.3	10.0.2.15	34.107.243.93	Client Hello
37	2.38978…	0.00146…	60	TCP	34.107.243.93	10.0.2.15	443 → 50444 [ACK] Seq=1 Ack=1231 Win=65535 Len=0
38	2.47246…	0.08268…	272	TLSv1.3	34.107.243.93	10.0.2.15	Server Hello, Change Cipher Spec, Application Data
39	2.47250…	0.00004…	54	TCP	10.0.2.15	34.107.243.93	50444 → 443 [ACK] Seq=1231 Ack=219 Win=64022 Len=0
40	2.47609…	0.00359…	118	TLSv1.3	10.0.2.15	34.107.243.93	Change Cipher Spec, Application Data
41	2.47790…	0.00180…	60	TCP	34.107.243.93	10.0.2.15	443 → 50444 [ACK] Seq=219 Ack=1295 Win=65535 Len=0
42	2.49120…	0.01330…	678	TLSv1.3	10.0.2.15	34.107.243.93	Application Data
43	2.49291…	0.00171…	60	TCP	34.107.243.93	10.0.2.15	443 → 50444 [ACK] Seq=219 Ack=1919 Win=65535 Len=0

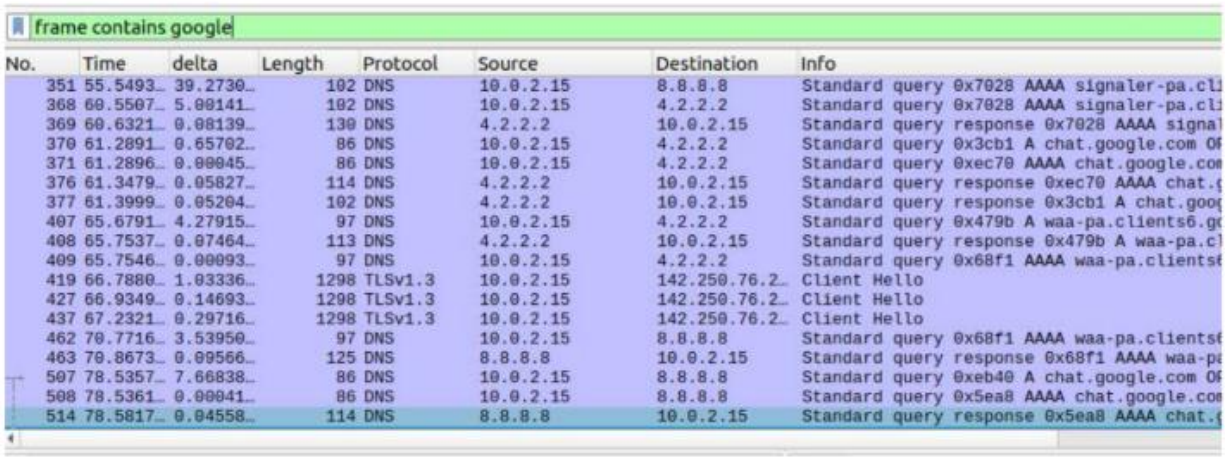

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
(!(ip.src == 10.0.2.15)) && !(ip.dst == 34.107.243.93)
tcp syn ecn
No. Time delta Length Protocol Source Destination Info
1 0.00000… 0.00000… 60 TCP 142.250.183.206 10.0.2.15 443 → 54274 [ACK] Seq=1 Ack=1 Win=65535 Len=0
2 0.00000… 0.00000… 60 TCP 142.250.183.206 10.0.2.15 443 → 54274 [ACK] Seq=1 Ack=276 Win=65535 Len=0
3 0.00015… 0.00015… 60 TCP 142.250.183.206 10.0.2.15 443 → 54274 [ACK] Seq=1 Ack=1736 Win=65535 Len=0
4 0.00015… 0.00000… 60 TCP 142.250.183.206 10.0.2.15 443 → 54274 [ACK] Seq=1 Ack=3196 Win=65535 Len=0
5 0.00015… 0.00000… 60 TCP 142.250.183.206 10.0.2.15 443 → 54274 [ACK] Seq=1 Ack=3679 Win=65535 Len=0
7 1.40064… 1.40048… 436 DNS 8.8.8.8 10.0.2.15 Standard query response 0x47d0 AAAA connectivity-check.ubuntu.com AAAA 262
8 2.20669… 0.80605… 571 TLSv1.2 142.251.42.110 10.0.2.15 Application Data, Application Data, Application Data, Application Data
10 2.20789… 0.00120… 60 TCP 142.251.42.110 10.0.2.15 443 → 58886 [ACK] Seq=518 Ack=40 Win=65535 Len=0
11 2.24300… 0.03510… 517 TLSv1.2 142.251.42.110 10.0.2.15 Application Data, Application Data, Application Data
13 2.27400… 0.03100… 60 TCP 142.251.42.110 10.0.2.15 443 → 58886 [ACK] Seq=981 Ack=795 Win=65535 Len=0
14 2.54220… 0.26820… 266 TLSv1.2 142.251.42.110 10.0.2.15 Application Data, Application Data
16 4.20918… 1.66688… 78 TLSv1.2 34.107.243.93 10.0.2.15 Application Data
18 4.21007… 0.00089… 60 TCP 34.107.243.93 10.0.2.15 443 → 50444 [ACK] Seq=25 Ack=29 Win=65535 Len=0
22 4.66326… 0.45318… 192 DNS 8.8.8.8 10.0.2.15 Standard query response 0xa218 A chat.google.com A 142.251.42.110 OPT
24 4.67973… 0.01647… 1286 TLSv1.2 142.250.183.206 10.0.2.15 Application Data, Application Data, Application Data, Application Data
26 4.68671… 0.00098… 60 TCP 142.250.183.206 10.0.2.15 443 → 54274 [ACK] Seq=1233 Ack=3718 Win=65535 Len=0
27 4.70550… 0.02478… 114 DNS 8.8.8.8 10.0.2.15 Standard query response 0xe20e AAAA chat.google.com AAAA 2404:6800:4009:82
30 7.68778… 2.98227… 60 TCP 142.250.183.163 10.0.2.15 443 → 47250 [ACK] Seq=1 Ack=98 Win=65535 Len=0


ip.addr eq 10.0.2.15 and ip.addr eq 142.250.199.138
tcp syn ecn
No. Time delta Length Protocol Source Destination Info
79 20.9698… 0.00000… 1399 QUIC 10.0.2.15 142.250.199… 0-RTT, DCID=8881007ebc5d60b2ce92d36c7e540d41a09662, SCID=9b05a9
83 21.0151… 0.04525… 74 TCP 10.0.2.15 142.250.199… 38162 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=7218617
84 21.0664… 0.05133… 1399 QUIC 10.0.2.15 bc5d60b2ce92d36c7e540d41a09662, SCID=9b05a9, PKN: 1,
85 21.0665… 0.00007… 1399 QUIC 10.0.2.15 bc5d60b2ce92d36c7e540d41a09662, SCID=9b05a9, PKN: 2,
90 21.2649… 0.19846… 1399 QUIC 10.0.2.15 bc5d60b2ce92d36c7e540d41a09662, SCID=9b05a9, PKN: 3,
91 21.2658… 0.00009… 1399 QUIC 10.0.2.15 bc5d60b2ce92d36c7e540d41a09662, SCID=9b05a9, PKN: 4,
92 21.2666… 0.00160… 74 TCP 10.0.2.15 0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=7218642
93 21.3591… 0.09247… 1399 QUIC 142.250.…], DCID=9b05a9
94 21.3859… 0.02680… 1399 QUIC 10.0.2.15], DCID=e881007ebc5d60b2
95 21.3862… 0.00032… 1399 QUIC 10.0.2.15], DCID=e881007ebc5d60b2
96 21.4049… 0.01866… 765 QUIC 10.0.2.15], DCID=e881007ebc5d60b2
97 21.4419… 0.03700… 165 QUIC 142.250.…], DCID=9b05a9
98 21.4427… 0.00081… 1399 QUIC 142.250.…], DCID=9b05a9
99 21.4435… 0.00077… 72 QUIC 142.250.…], DCID=9b05a9
100 21.4600… 0.01654… 74 QUIC 10.0.2.15], DCID=e881007ebc5d60b2
105 21.5127… 0.05267… 1292 QUIC 142.250.…], DCID=9b05a9
106 21.5133… 0.00059… 217 QUIC 142.250.…], DCID=9b05a9
107 21.5344… 0.02107… 78 QUIC 10.0.2.15], DCID=e881007ebc5d60b2
Frame 83: 74 bytes on wire (592 bits), 74 bytes captu
Ethernet II, Src: PcsCompu_b8:37:09 (08:00:27:b8:37:0
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 142
Transmission Control Protocol, Src Port: 38162, Dst P
Source Port: 38162
Destination Port: 443
[Stream index: 5]
[Conversation completeness: Complete, WITH_DATA (3
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
08 00 27 b8 37 09 08 00 45 00 RT 5 ' 7 E
40 06 a8 1a 0a 00 02 0f 8e fa <0 @ @
fc f2 27 32 00 00 00 00 a0 02 '2
02 04 05 b4 04 02 08 0a 04 4d b M
01 03 03 07 y<
Mark/Unmark Packet Ctrl+M
Ignore/Unignore Packet Ctrl+D
Set/Unset Time Reference Ctrl+T
Time Shift… Ctrl+Shift+T
Packet Comments
Edit Resolved Name
Apply as Filter
Prepare as Filter
Conversation Filter
Colorize Conversation
SCTP
Follow
Copy
Protocol Preferences
Decode As…
Show Packet in New Window


File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.port in {80}
tcp s
No. Time delta Length Protocol Source Destination Info
3 2.44203… 0.00000… 54 TCP 10.0.2.15 142.250.192… 39034 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
4 2.44236… 0.00032… 60 TCP 142.250.192.131 10.0.2.15 [TCP ACKed unseen segment] 80 → 39034 [ACK] Seq=1 Ack=2 Win=65535 L
63 4.74607… 2.30371… 54 TCP 10.0.2.15 142.250.192… 46256 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
64 4.74655… 0.00047… 60 TCP 142.250.192.131 10.0.2.15 [TCP ACKed unseen segment] 80 → 46256 [ACK] Seq=1 Ack=2 Win=65535 L
178 12.6829… 7.93548… 54 TCP 10.0.2.15 142.250.192… [TCP Dup ACK 3#1] 39034 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
179 12.6823… 0.00033… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 4#1] [TCP ACKed unseen segment] 80 → 39034 [ACK] Seq=1
180 14.9861… 2.30374… 54 TCP 10.0.2.15 142.250.192… [TCP Dup ACK 63#1] 46256 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
181 14.9865… 0.00048… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 64#1] [TCP ACKed unseen segment] 80 → 46256 [ACK] Seq=
260 22.9221… 7.93556… 54 TCP 10.0.2.15 142.250.192… [TCP Dup ACK 3#2] 39034 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
261 22.9225… 0.00038… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 4#2] [TCP ACKed unseen segment] 80 → 39034 [ACK] Seq=1
272 25.2260… 2.30349… 54 TCP 10.0.2.15 142.250.192… [TCP Dup ACK 63#2] 46256 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
273 25.2264… 0.00035… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 64#2] [TCP ACKed unseen segment] 80 → 46256 [ACK] Seq=
305 33.1621… 7.93576… 54 TCP 10.0.2.15 142.250.192… [TCP Dup ACK 3#3] 39034 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
306 33.1627… 0.00056… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 4#3] [TCP ACKed unseen segment] 80 → 39034 [ACK] Seq=1
310 35.4661… 2.30340… 54 TCP 10.0.2.15 142.250.192… [TCP Dup ACK 63#3] 46256 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
311 35.4669… 0.00080… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 64#3] [TCP ACKed unseen segment] 80 → 46256 [ACK] Seq=
331 43.4027… 7.93584… 54 TCP 10.0.2.15 142.250.192… [TCP Dup ACK 3#4] 39034 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
332 43.4032… 0.00045… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 4#4] [TCP ACKed unseen segment] 80 → 39034 [ACK] Seq=1


frame contains sbi
No. Time delta Length Protocol Source Destination Info
1631 290.857… 0.00000… 91 DNS 10.0.2.15 8.8.8.8 Standard query 0x7f86 A retail.onlinesbi.sbi OPT
1632 290.981… 0.12334… 91 DNS 10.0.2.15 8.8.8.8 Standard query 0x14e8 AAAA retail.onlinesbi.sbi OPT
1655 295.863… 4.88265… 91 DNS 10.0.2.15 4.2.2.2 Standard query 0x7f86 A retail.onlinesbi.sbi OPT
1656 295.986… 0.12245… 91 DNS 10.0.2.15 4.2.2.2 Standard query 0x14e8 AAAA retail.onlinesbi.sbi OPT

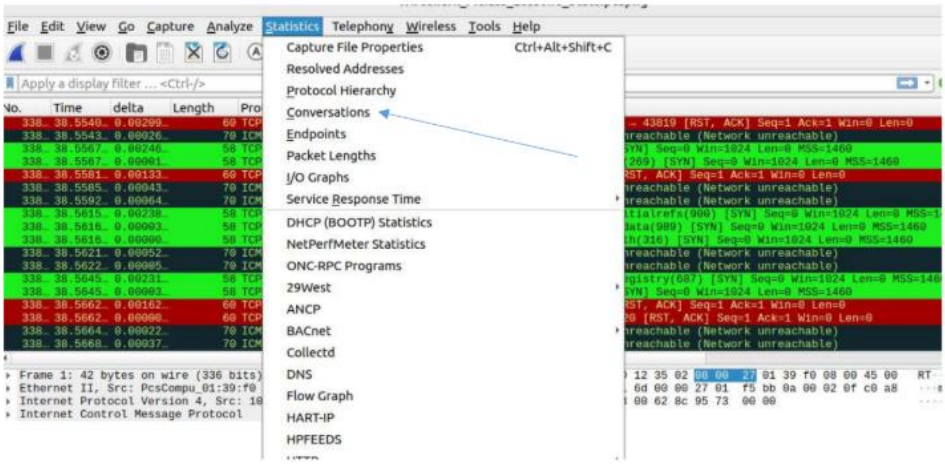

frame contains firefox
No. Time delta Length Protocol Source Destination Info
606 83.4648… 0.00000… 104 DNS 10.0.2.15 8.8.8.8 Standa
611 83.5042… 0.03932… 185 DNS 8.8.8.8 10.0.2.15 Standa
612 83.5051… 0.00096… 125 DNS 10.0.2.15 8.8.8.8 Standa
619 83.5318… 0.02665… 153 DNS 8.8.8.8 10.0.2.15 Standa
635 83.6708… 0.13904… 735 TLSv1.3 10.0.2.15 34.149.97.1 Client

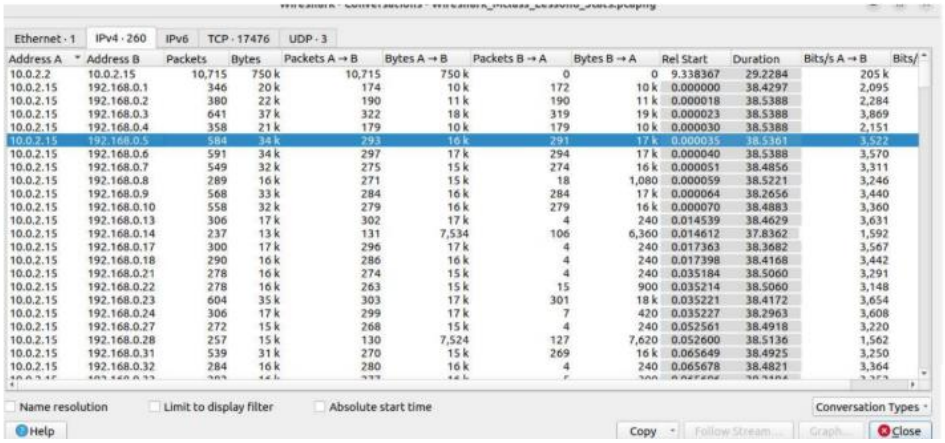

frame contains google
No. Time delta Length Protocol Source Destination Info
351 55.5493… 39.2730… 102 DNS 10.0.2.15 8.8.8.8 Standard query 0x7028 AAAA signaler-pa.cl
368 60.5507… 5.00141… 102 DNS 10.0.2.15 4.2.2.2 Standard query 0x7028 AAAA signaler-pa.cl
369 60.6321… 0.08139… 130 DNS 4.2.2.2 10.0.2.15 Standard query response 0x7028 AAAA signal
370 61.2891… 0.65702… 86 DNS 10.0.2.15 4.2.2.2 Standard query 0x3cb1 A chat.google.com O
371 61.2896… 0.00045… 86 DNS 10.0.2.15 4.2.2.2 Standard query 0xec70 AAAA chat.google.co
376 61.3479… 0.05827… 114 DNS 4.2.2.2 10.0.2.15 Standard query response 0xec70 AAAA chat.
377 61.3999… 0.05204… 102 DNS 4.2.2.2 10.0.2.15 Standard query response 0x3cb1 A chat.goo
407 65.6791… 4.27915… 97 DNS 10.0.2.15 4.2.2.2 Standard query 0x479b A waa-pa.clients6.g
408 65.7537… 0.07464… 113 DNS 4.2.2.2 10.0.2.15 Standard query response 0x479b A waa-pa.c
409 65.7546… 0.00093… 97 DNS 10.0.2.15 4.2.2.2 Standard query 0x68f1 AAAA waa-pa.clients
419 66.7880… 1.03336… 1298 TLSv1.3 10.0.2.15 142.250.76.2… Client Hello
427 66.9349… 0.14693… 1298 TLSv1.3 10.0.2.15 142.250.76.2… Client Hello
437 67.2321… 0.29716… 1298 TLSv1.3 10.0.2.15 142.250.76.2… Client Hello
462 70.7716… 3.53950… 97 DNS 10.0.2.15 8.8.8.8 Standard query 0x68f1 AAAA waa-pa.clients
463 70.8673… 0.09566… 125 DNS 8.8.8.8 10.0.2.15 Standard query response 0x68f1 AAAA waa-p
507 78.5357… 7.66838… 86 DNS 10.0.2.15 8.8.8.8 Standard query 0xeb40 A chat.google.com O
508 78.5361… 0.00041… 86 DNS 10.0.2.15 8.8.8.8 Standard query 0x5ea8 AAAA chat.google.co
514 78.5817… 0.04558… 114 DNS 8.8.8.8 10.0.2.15 Standard query response 0x5ea8 AAAA chat.

SET—11

WIRESHARK STATISTICS.

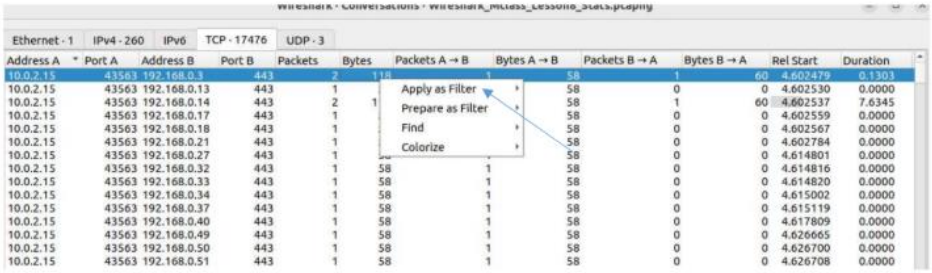
Go to:



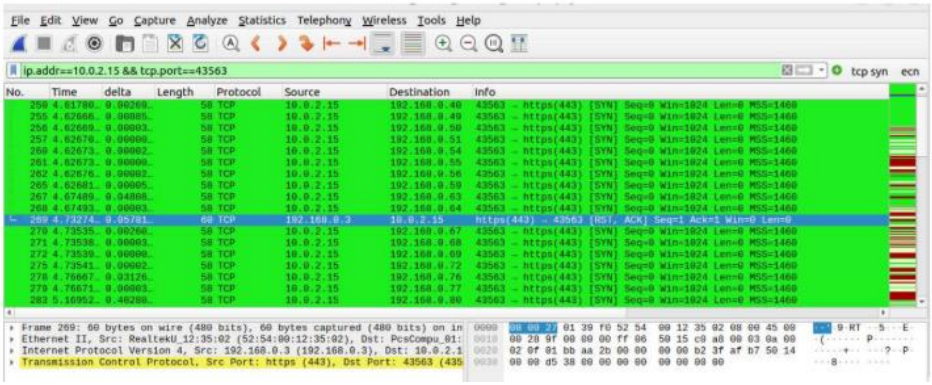


Address A	Address B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B
10.0.2.2	10.0.2.15	10,715	750 k	10,715	750 k	0	0	9.338367	29.2284	205 k
10.0.2.15	192.168.0.1	346	20 k	174	10 k	172	10 k	0.000000	38.4297	2,095
10.0.2.15	192.168.0.2	380	22 k	190	11 k	190	11 k	0.000018	38.5388	2,284
10.0.2.15	192.168.0.3	641	37 k	322	18 k	319	19 k	0.000023	38.5388	3,869
10.0.2.15	192.168.0.4	358	21 k	179	10 k	179	10 k	0.000030	38.5388	2,151
10.0.2.15	192.168.0.5	584	34 k	293	16 k	291	17 k	0.000035	38.5361	3,522
10.0.2.15	192.168.0.6	591	34 k	297	17 k	294	17 k	0.000040	38.5388	3,570
10.0.2.15	192.168.0.7	549	32 k	275	15 k	274	16 k	0.000051	38.4856	3,311
10.0.2.15	192.168.0.8	289	16 k	271	15 k	18	1,080	0.000059	38.5221	3,246
10.0.2.15	192.168.0.9	568	33 k	284	16 k	284	17 k	0.000064	38.2656	3,440
10.0.2.15	192.168.0.10	558	32 k	279	16 k	279	16 k	0.000070	38.4883	3,360
10.0.2.15	192.168.0.13	306	17 k	302	17 k	4	240	0.014539	38.4629	3,631
10.0.2.15	192.168.0.14	237	13 k	131	7,534	106	6,360	0.014612	37.8362	1,592
10.0.2.15	192.168.0.17	300	17 k	296	17 k	4	240	0.017363	38.3682	3,567
10.0.2.15	192.168.0.18	290	16 k	286	16 k	4	240	0.017398	38.4168	3,442
10.0.2.15	192.168.0.21	278	16 k	274	15 k	4	240	0.035184	38.5060	3,291
10.0.2.15	192.168.0.22	278	16 k	263	15 k	15	900	0.035214	38.5060	3,148
10.0.2.15	192.168.0.23	604	35 k	303	17 k	301	18 k	0.035221	38.4172	3,654
10.0.2.15	192.168.0.24	306	17 k	299	17 k	7	420	0.035227	38.2963	3,608
10.0.2.15	192.168.0.27	272	15 k	268	15 k	4	240	0.052561	38.4918	3,220
10.0.2.15	192.168.0.28	257	15 k	130	7,524	127	7,620	0.052600	38.5136	1,562
10.0.2.15	192.168.0.31	539	31 k	270	15 k	269	16 k	0.065649	38.4925	3,250
10.0.2.15	192.168.0.32	284	16 k	280	16 k	4	240	0.065678	38.4821	3,364

Adding filter from the conversation and viewing in the packet list:



It looks as follows:



Program to implement select socket system call.

```
#include <stdio.h>
#include <stdlib.h>
 #include <string.h>
 #include <unistd.h>
 #include <arpa/inet.h>
#include <sys/socket.h>
 #include <sys/select.h>

int main()

{ int sockfd, connfd;
 struct sockaddr_in servaddr;
fd_set readfds;
char buffer[1024];

sockfd = socket(AF_INET, SOCK_STREAM, 0);
servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
servaddr.sin_port = htons(5000);
```

```
bind(sockfd, (struct sockaddr*)&servaddr, sizeof(servaddr));
listen(sockfd, 1);
connfd = accept(sockfd, NULL, NULL);

while (1) {
   FD_ZERO(&readfds);
   FD_SET(0, &readfds);
   FD_SET(connfd, &readfds);

   select(connfd + 1, &readfds, NULL, NULL, NULL);

   if (FD_ISSET(0, &readfds)) {
      memset(buffer, 0, sizeof(buffer));
      read(0, buffer, sizeof(buffer));
      write(connfd, buffer, strlen(buffer));
   }

   if (FD_ISSET(connfd, &readfds)) {
      memset(buffer, 0, sizeof(buffer));
      read(connfd, buffer, sizeof(buffer));
      printf("Received: %s\n", buffer);
   }
}

close(connfd);
close(sockfd);
return 0;


}
```